Mikroprocesorová technika

Prednáška č.6: A/D prevodník a
spracovanie analógových signálov

prof. Ing. Ivan Virgala, PhD.

Technická univerzita v Košiciach
Strojnícka fakulta

Katedra priemyselnej automatizácie a mechatroniky
ARM-lab (Applied Robotics and Mechatronics Laboratory)



A/D prevod a analógové meranie

Od fyzikálnej veličiny k číselnej hodnote v mikrokontroléri

Obsah prednášky

- princíp vzorkovania, kvantovania a kódovania analógového signálu,
- rozlíšenie 10-bitového A/D prevodníka a význam referenčného napätia,
- periféria ADC mikrokontroléra `ATmega328P` a jej registre,
- jednorazový prevod, voľný beh a prerušenie po dokončení prevodu,
- programovanie ADC v jazyku C a bezpečný prepočet výsledku na napätie,
- základné techniky potlačenia šumu a spracovania meraných údajov,
- aplikácia s analógovým snímačom vzdialenosti.

Využitie A/D prevodu



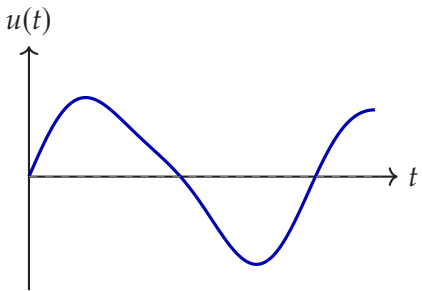
Typické analógové snímače

- teplota, tlak, sila, poloha,
- intenzita svetla,
- prúd a napätie,
- analógový výstup snímača vzdialenosti.

Úloha ADC

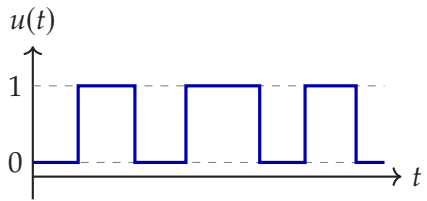
Prevodník nepriraďuje napätiu fyzikálny význam. Vytvorí číselný kód, ktorý program prepočíta na volty, stupne, newtony alebo inú veličinu.

Analógový signál



Signál môže nadobúdať spojito meniace sa hodnoty.

Digitálny signál



Signál nadobúda diskrétne logické úrovne 0 a 1.

Analógový a digitálny opis signálu

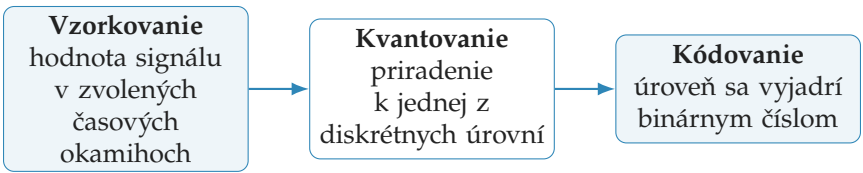
Analógový signál

- napätie sa môže meniť spojito v čase,
- medzi dvoma hodnotami existuje množstvo ďalších hodnôt,
- typický výstup snímača je napr. 0,8 V; 2,1 V; 3,7 V.

Digitálna reprezentácia

- ADC meria signál v konkrétnych časových okamihoch,
- výsledkom A/D prevodu je jedno z konečného počtu digitálnych čísel,
- 10-bitový ADC poskytuje digitálnu hodnotu v rozsahu 0 . . . 1023.

Tri kroky A/D prevodu



Dôsledok

Digitálny výsledok je diskrétny v čase aj v amplitúde. Informácia medzi dvoma vzorkami a medzi dvoma kvantizačnými úrovňami sa priamo nezachytí.

Rozlíšenie, referencia a vzorkovanie

Ako sa zo vstupného napätia vytvorí 10-bitový výsledok

Rozlíšenie 10-bitového prevodníka

Pre n-bitový prevodník je počet možných digitálnych kódov

$$N = 2^n.$$

Pre `ATmega328P` platí $n = 10$:

$$2^{10} = 1024.$$

Výsledný rozsah kódov

$$0, 1, 2, \ldots, 1023$$

Dôležité

1024 úrovní neznamená, že najväčšie číslo je 1024. Pri 10 bitoch je najväčší binárny kód $1111111111_2 = 1023$.

Najmenší krok prevodníka – LSB

Najmenší ideálny napäťový krok je

$$V_{LSB} = \frac{V_{REF}}{2^n}.$$

Referencia 5 V

$$V_{LSB} = \frac{5}{1024} \approx 4{,}88\,\text{mV}$$

Jedna zmena kódu približne o 1 predstavuje zmenu vstupného napätia o 4,88 mV.

Interná referencia 1,1 V

$$V_{LSB} = \frac{1{,}1}{1024} \approx 1{,}07\,\text{mV}$$

Pri menšom meracom rozsahu získame jemnejšie rozlíšenie napätia.

Prevod vstupného napätia na digitálny kód

Pre ideálny A/D prevodník s rozsahom 0 až V_{REF} možno výsledok približne vyjadriť

$$ADC \approx \frac{V_{IN}}{V_{REF}} 2^{10}$$

Príklad

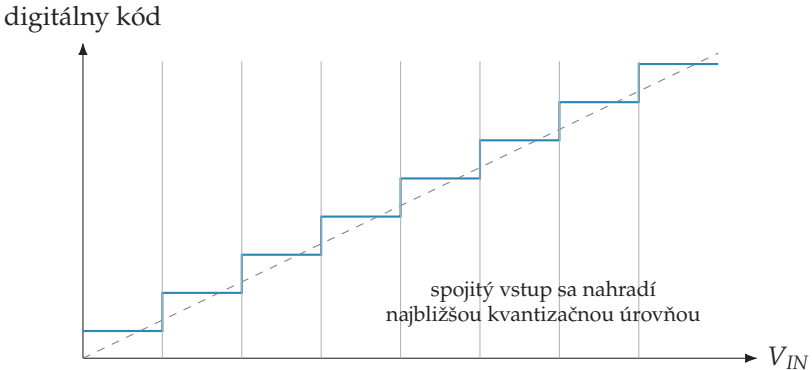
Pre $V_{REF} = 5\,\text{V}$ a $V_{IN} = 2{,}5\,\text{V}$:

$$ADC \approx \frac{2{,}5}{5} \cdot 1024 = 512$$

Obmedzenie rozsahu

Výsledný register je 10-bitový. Hodnota je preto obmedzená na rozsah $0 \ldots 1023$. Napätie nad referenciou nevedie k väčšiemu číslu, ale k saturácii výsledku.

Kvantizačná chyba



Ideálny prípad

Kvantovanie vytvára nevyhnutnú neistotu približne $\pm 0{,}5\,LSB$. Skutočný prevodník má navyše offset, chybu zosilnenia, nelinearitu a šum.

Referenčné napätie určuje merací rozsah

Väčšia referencia

- väčší merací rozsah vstupného napätia,
- väčší krok LSB,
- menšie napäťové rozlíšenie.

Menšia referencia

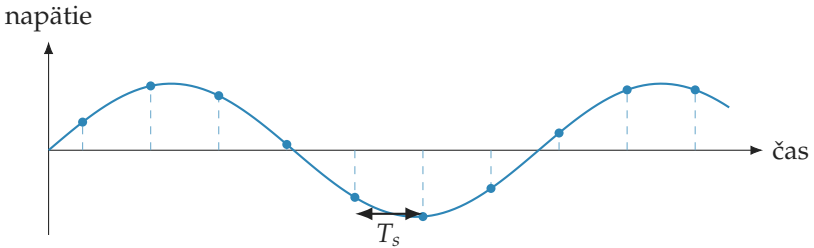
- menší merací rozsah,
- menší krok LSB,
- jemnejšie napäťové rozlíšenie.

Podmienka

Vstupné napätie musí zostať v povolenom rozsahu vstupu mikrokontroléra. Referencia nie je ochranný prvok proti prepätiu na pine.

Ak ADC vykonáva meranie každých T_s sekúnd, vzorkovacia frekvencia je

$$f_s = \frac{1}{T_s}.$$



Nyquistova podmienka a aliasing

Aby bolo možné časovo premenlivý signál správne reprezentovať, musí byť vzorkovanie dostatočne rýchle. Základná podmienka je

$$f_s > 2f_{max}$$

Ak je podmienka splnená

Vzorky nesú dostatok informácie o najvyššej frekvenčnej zložke sledovaného signálu.

Ak je vzorkovanie príliš pomalé

Vzniká **aliasing**: rýchla zložka sa vo vzorkovaných dátach môže javiť ako iná, nižšia frekvencia.

Praktické riešenie

Pred ADC sa pri rýchlo sa meniacich signáloch používa dolnopriepustný filter, ktorý potlačí príliš vysoké frekvencie, aby ich ADC nespracoval nesprávne.

ADC v ATmega328P

Multiplexor, referencia, hodinový signál a riadiace registre

Základné vlastnosti ADC v ATmega328P

Prevodník

- 10-bitový ADC s postupnou aproximáciou,
- jednokanálové analógové vstupy,
- jednorazový prevod a automatické spúšťanie,
- prerušenie po dokončení prevodu.

Kanály

- 6 kanálov v puzdre PDIP,
- 8 kanálov v TQFP/QFN puzdrách,
- Arduino UNO R3 štandardne sprístupňuje `A0-A5`.

Piny Arduino UNO R3

`A0-A5` zodpovedajú pinom `PC0/ADC0` až `PC5/ADC5`. Piny `PC4` a `PC5` majú zároveň alternatívne funkcie SDA a SCL.

Základné bloky A/D prevodníka

Sample & Hold

- Vstupné napätie sa na krátky čas uloží na vnútorný kondenzátor.
- ADC tak pracuje so stabilnou hodnotou vstupného signálu.

SAR-Successive Approximation Reg.

- Prevodník postupne určuje jednotlivé bity výsledku, začínajúc najvyšším bitom.
- Výsledok: digitálna hodnota vstupného napätia.

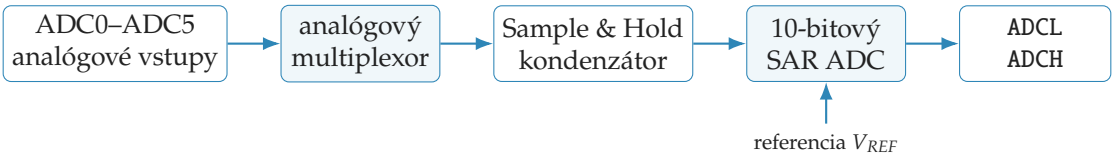
Výsledok A/D prevodu

Výsledok obsahuje 10 bitov. Keďže mikrokontrolér pracuje s 8-bitovými registrami, výsledok je rozdelený do dvoch registrov.

Registre výsledku

- `ADCL` – dolná časť výsledku,
- `ADCH` – horná časť výsledku.

Vnútorná dátová cesta ADC



Význam multiplexora

Mikrokontrolér má jeden prevodník, pred ktorý je zaradený multiplexor. V jednom okamihu sa preto prevádza iba jeden zvolený analógový kanál.

Referenčné napätie – voľba REFS1:0

REFS1	REFS0	Referencia ADC
0	0	externé napätie na pine AREF; interná referencia je vypnutá
0	1	AVCC; na AREF sa používa externý blokovací kondenzátor (pomáha udržať napájanie stabilné a odfiltrovať vysokofrekvenčné rušenie)
1	0	rezervované
1	1	interná referencia približne 1,1 V; kondenzátor na AREF

AREF

Na pin AREF nepripájame svojvoľne napätie, ak je zvolená interná referencia. Spôsob zapojenia musí zodpovedať zvolenému režimu referencie.

Register ADMUX

7	6	5	4	3	2	1	0
REFS1	REFS0	ADLAR	–	MUX3	MUX2	MUX1	MUX0

- `REFS1:0` – výber referenčného napätia,
- `ADLAR` – zarovnanie 10-bitového výsledku v registroch `ADCH:ADCL`,
- `MUX3:0` – výber meraného analógového vstupu.

Príklad

Pre `ADC0` a referenciu `AVCC`: $REFS1 : 0 = 01$ a $MUX3 : 0 = 0000$.

Výber kanála pomocou MUX3:0

`MUX3:0`	Kanál	Pin na Arduino UNO R3
0000	ADC0	A0 / PC0
0001	ADC1	A1 / PC1
0010	ADC2	A2 / PC2
0011	ADC3	A3 / PC3
0100	ADC4	A4 / PC4
0101	ADC5	A5 / PC5

Pri zmene kanála

Multiplexor pripojí k Sample & Hold obvodu iný vstup. Pri vysokej impedancii zdroja môže byť potrebný čas na ustálenie napätia na vzorkovacom kondenzátore.

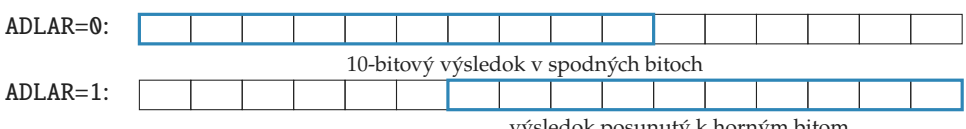
Zarovnanie výsledku – ADLAR

ADLAR = 0

Výsledok je zarovnaný doprava. Používa sa pri plnom 10-bitovom meraní; čítajú sa registre ADCL a ADCH.

ADLAR = 1

Výsledok je zarovnaný doľava. Ak stačí 8-bitové rozlíšenie, možno použiť iba register ADCH.



ADLAR – zarovnanie výsledku ADC

ADC výsledok:

$682_{10} = 1010101010_2$

ADLAR = 0 – doprava

ADCH	ADCL
00000010	10101010

Celý 10-bitový výsledok sa číta z `ADCH` a `ADCL`.

ADLAR = 1 – doľava

ADCH	ADCL
10101010	10000000

Horných 8 bitov výsledku je priamo v `ADCH`.

Dôležité

`ADLAR` nemení nameranú hodnotu, iba jej uloženie v registroch. Zarovnanie doprava je vhodné pre celý 10-bitový výsledok, zarovnanie doľava pri použití iba 8 najvýznamnejších bitov.

Register ADCSRA

Bit	Názov	Význam
7	ADEN	zapnutie ADC
6	ADSC	spustenie prevodu; počas prevodu zostáva nastavený
5	ADATE	povolenie automatického spúšťania
4	ADIF	príznak dokončenia prevodu
3	ADIE	povolenie prerušenia po dokončení prevodu
2:0	ADPS2:0	voľba preddeličky hodinového signálu ADC

Pre maximálne 10-bitové rozlíšenie sa používa hodinová frekvencia ADC približne v rozsahu

$$50\,\text{kHz} \leq f_{ADC} \leq 200\,\text{kHz}.$$

Arduino UNO R3 – 16 MHz

Pri preddeličke 128:

$$f_{ADC} = \frac{16\,\text{MHz}}{128} = 125\,\text{kHz}.$$

Hodnota 125 kHz je v odporúčanom rozsahu.

ADPS2	ADPS1	ADPS0	Faktor
0	0	0	2
0	0	1	2
0	1	0	4
0	1	1	8
1	0	0	16
1	0	1	32
1	1	0	64
1	1	1	128

Pre 16 MHz

Na plné 10-bitové meranie je prirodzená voľba `ADPS2:0 = 111`, teda delenie 128.

Čas jedného prevodu

Bežný jednokanálový prevod potrebuje 13 períód hodinového signálu ADC.
Pri $f_{ADC} = 125\,\text{kHz}$:

$$T_{ADC} = \frac{1}{125000} = 8\,\mu\text{s}$$

$$T_{conv} = 13 \cdot 8\,\mu\text{s} = 104\,\mu\text{s}.$$

Praktický dôsledok

Pri $f_{ADC} = 125\,\text{kHz}$ trvá jeden bežný prevod približne $104\,\mu s$, takže ADC môže vykonať približne 9600 prevodov za sekundu. Prvý prevod po zapnutí ADC trvá dlhšie, pretože sa musí najprv ustáliť vnútorná analógová časť prevodníka.

Registre výsledku ADCL a ADCH

10-bitový výsledok je uložený v dvojici 8-bitových registrov.

Poradie čítania

Pri samostatnom čítaní registrov sa číta najprv `ADCL` a potom `ADCH`. Čítanie `ADCL` uzamkne výsledok až do prečítania `ADCH`, aby oba bajty patrili k tomu istému prevodu.

Čítanie výsledku ADC v jazyku C

V programe možno použiť 16-bitový názov registra `ADC`, ktorý reprezentuje dvojicu registrov `ADCL` a `ADCH`. Pri jeho použití sa oba registre prečítajú v správnom poradí:

```
uint16_t value = ADC;
```

Jednorazový prevod a automatické spúšťanie

Single Conversion

1. nastaví sa `ADSC`,
2. vykoná sa jeden prevod,
3. po dokončení sa `ADSC` vynuluje,
4. ďalší prevod spúšťa program znovu.

Auto Trigger

- `ADATE = 1`,
- zdroj spustenia určuje `ADTS2:0` v registri `ADCSRB`,
- zdrojom môže byť napr. voľný beh, časovač alebo externá udalosť.

Princíp

V režime voľného behu sa po dokončení jedného prevodu automaticky spustí ďalší. ADC tak vytvára súvislý prúd vzoriek bez opakovaného nastavovania `ADSC` programom.



Použitie

Vhodné pre pravidelné kontinuálne meranie, ak program vie výsledky spracovať dostatočne rýchlo.

Prerušenie po dokončení prevodu

Ak je `ADIE = 1` a globálne prerušenia sú povolené, dokončenie prevodu môže vyvolať obsluhu

`ISR(ADC_vect)`.

Priebeh

1. ADC dokončí prevod,
2. výsledok sa uloží do `ADCL/ADCH`,
3. nastaví sa príznak `ADIF`,
4. procesor vykoná vektor `ADC_vect`.

Digitálny vstup na analógovom pine – DIDR0

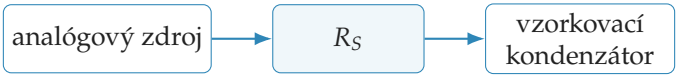
Digital Input Disable Register 0

Bity `ADC0D-ADC5D` vypínajú digitálny vstup na analógových pinoch.

- znižuje sa zbytočná spotreba digitálneho vstupu,
- pri analógovom meraní sa môže znížiť rušenie,
- pin zostáva dostupný pre analógový multiplexor ADC.

Príklad

Ak používame iba `ADC0`, môžeme nastaviť `ADC0D = 1` v registri `DIDR0`.



Odporúčanie

ADC je optimalizovaný pre zdroje s výstupnou impedanciou približne 10 k˙ alebo menšou. Vyššia impedancia predlžuje nabíjanie vzorkovacieho kondenzátora a môže zhoršiť presnosť.

Potlačenie šumu pri analógovom meraní

Hardvérové opatrenia

- krátke analógové signálové cesty,
- oddelenie od rýchlo sa meniacich digitálnych vodičov,
- správne blokovanie `AVCC` a `AREF`,
- vhodná topológia zeme a napájania,
- nízka impedancia analógového zdroja.

Softvérové opatrenia

- priemerovanie viacerých vzoriek,
- vhodná vzorkovacia frekvencia,
- meranie v čase menšieho digitálneho rušenia,
- podľa potreby režim ADC Noise Reduction.

Programovanie ADC v jazyku C

Inicializácia, čítanie kanála a prepočet výsledku

Dátové typy pre výsledok ADC

10-bitový výsledok potrebuje viac ako 8 bitov.

`uint8_t`

Rozsah 0 . . . 255. Na plný 10-bitový výsledok nestačí.

`uint16_t`

Rozsah 0 . . . 65535. Bez problémov uloží hodnotu 0 . . . 1023.

Prečo typy `uint16_t`, `uint32_t`

Typy z hlavičky `stdint.h` majú presne definovanú šírku, čo je pri práci s registrami a výpočtami v embedded C výhodné.

Inicializácia ADC

```c
#include <avr/io.h>
#include <stdint.h>

void adc_init(void)
{
    // referencia AVCC, kanal ADC0
    ADMUX = (1 << REFS0);

    // ADC zapnuty, prescaler 128
    ADCSRA = (1 << ADEN) |
             (1 << ADPS2) |
             (1 << ADPS1) |
             (1 << ADPS0);

    // vypnut digitalny vstup na ADC0
    DIDR0 = (1 << ADC0D);
}
```

Interpretácia inicializácie ADC

- `REFS0` = 1 – referencia `AVCC`,
- `MUX3:0` = `0000` – zvolený kanál `ADC0`,
- `ADEN` = 1 – ADC je zapnutý,
- `ADPS2:0` = 111 – preddelička 128,
- pri $f_{CPU} = 16\,\text{MHz}$ vznikne $f_{ADC} = 125\,\text{kHz}$,
- `ADC0D` = 1 – digitálny vstupný buffer na A0 je vypnutý.

Jednorazové čítanie zvoleného kanála

```
uint16_t adc_read(uint8_t channel)
{
    channel &= 0x07;

    // zachovaj referenciu, zmen iba MUX bity
    ADMUX = (ADMUX & 0xF0) | channel;

    // spusti jeden prevod
    ADCSRA |= (1 << ADSC);

    // cakaj na dokoncenie prevodu
    while (ADCSRA & (1 << ADSC)) {
    }

    return ADC;
}
```

Ako pracuje funkcia adc_read()

1. číslo kanála sa obmedzí na platné bity,
2. v `ADMUX` sa zmení iba výber kanála,
3. nastavením `ADSC` sa spustí prevod,
4. program čaká, kým hardvér `ADSC` po dokončení vynuluje,
5. vráti sa 10-bitový výsledok z registra `ADC`.

Charakter funkcie

Ide o **blokujúce meranie**: procesor počas prevodu čaká v cykle a nevykonáva inú časť hlavného programu.

Najjednoduchšie meranie na A0

```c
int main(void)
{
    uint16_t adc_value;

    adc_init();

    while (1)
    {
        adc_value = adc_read(0);   // A0 / ADC0

        // dalsie spracovanie hodnoty
    }
}
```

Prepočet kódu ADC na napätie

Pri ideálnom prevodníku možno napätie odhadnúť

$$V_{IN} \approx ADC \cdot \frac{V_{REF}}{1024}.$$

Pre referenciu 5 V:

$$U_{mV} \approx ADC \cdot \frac{5000}{1024}.$$

Príklad

Ak je $ADC = 512$, potom

$$U \approx 512 \cdot \frac{5000}{1024} = 2500\,\text{mV}.$$

Pozor na pretečenie pri výpočte v C

Na AVR má typ `int` typicky 16 bitov. Výraz

```
1 voltage_mv = adc_value * 5000 / 1024;
```

môže pretekať už pri medzivýsledku `adc_value * 5000`.

Bezpečnejší výpočet

```
1 uint32_t voltage_mv;
2
3 voltage_mv = ((uint32_t)adc_value * 5000UL) / 1024UL;
```

Prečo používame explicitnú konverziu typu

V zápise

```
(uint32_t)adc_value
```

sa 16-bitová hodnota pred násobením prevedie na 32-bitové celé číslo.

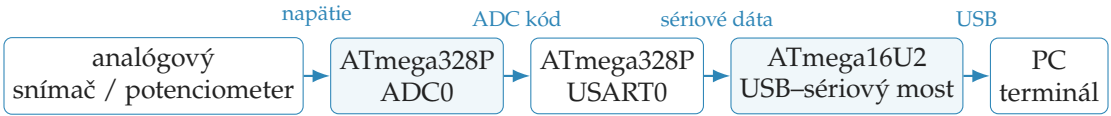
Dôsledok

Násobenie sa vykoná v 32-bitovej aritmetike, takže medzivýsledok do približne 5,1 milióna nepretečie.

Embedded C

Pri celočíselných výpočtoch je potrebné sledovať nielen výsledný typ premennej, ale aj typy medzivýrazov.

Merací reťazec ADC – USART – PC



Princíp príkladu

Mikrokontrolér periodicky zmeria napätie na `ADC0`, vypočíta jeho hodnotu v milivoltoch, vytvorí textový reťazec a odošle ho cez `USART0`. PC prijaté riadky zobrazuje v sériovom termináli.

Arduino UNO R3

Pri komunikácii cez USB konektor dosky nie je potrebný externý USB–UART prevodník. Prevod medzi USB a USART0 zabezpečuje mikrokontrolér ATmega16U2 na doske UNO R3.

Parametre komunikácie

- prenosová rýchlosť: **9600 bit/s**,
- dátové bity: **8**,
- parita: **žiadna**,
- stop bit: **1**,
- riadenie toku: **žiadne**.

Terminálový program

Možno použiť ľubovoľný sériový terminál, napríklad PuTTY, Tera Term alebo RealTerm. Zvolí sa sériový port priradený doske Arduino UNO R3 a rovnaké parametre ako v USART0.

Podmienka správneho prenosu

Nastavenie terminálu musí zodpovedať konfigurácii `USART0`. Pri rozdielnej baud rate alebo formáte rámca sa zobrazia nesprávne znaky alebo sa dáta neprijmú korektne.

Program: inicializácia ADC a USART0

```c
#define F_CPU 16000000UL
#include <avr/io.h>
#include <stdint.h>
#include <stdio.h>
#include <util/delay.h>
#include "usart0.h"

int main(void)
{
    char text[48];
    uint16_t adc_value;
    uint32_t voltage_mv;

    adc_init();
    usart0_init();          // 9600 bit/s, 8N1

    while (1)
    {
        // meranie a odoslanie - dalsi slajd
    }
}
```

Program: meranie a odosielanie do PC

```
while (1)
{
    adc_value = adc_read(0);        // ADC0 / A0
    voltage_mv = ((uint32_t)adc_value * 5000UL) / 1024UL;

    snprintf(text, sizeof(text),
             "ADC=%u, U=%lu mV\\r\\n",
             (unsigned)adc_value,
             (unsigned long)voltage_mv);

    usart0_send_string(text);
    _delay_ms(250);
}
```

Význam oneskorenia

Výpis odosielame štyrikrát za sekundu, aby bol terminál čitateľný. ADC samotný môže merať podstatne rýchlejšie.

Zobrazenie nameraných hodnôt v termináli

Príklad výstupu

```
ADC=204, U=996 mV
ADC=318, U=1552 mV
ADC=512, U=2500 mV
ADC=736, U=3593 mV
ADC=921, U=4497 mV
```

Čo sledujeme

- surový 10-bitový ADC kód,
- prepočet na napätie,
- zmenu hodnoty v reálnom čase,
- stabilitu a šum merania.

Praktické použitie

Takýto výpis je vhodný pri oživovaní snímača, kontrole kalibrácie alebo diagnostike meracieho reťazca. Neskôr možno rovnaké údaje ukladať alebo graficky spracovať v PC.

```
1 volatile uint16_t adc_value = 0;
2 volatile uint8_t adc_ready = 0;
3
4 ISR(ADC_vect)
5 {
6     adc_value = ADC;
7     adc_ready = 1;
8 }
```

Význam

ISR iba uloží výsledok a nastaví príznak. Výpočty, formátovanie textu a komunikácia cez USART patria do hlavného programu, nie do ISR.

```
1  void adc_interrupt_init(void)
2  {
3      ADMUX = (1 << REFS0);
4
5      ADCSRA = (1 << ADEN) |
6               (1 << ADIE) |
7               (1 << ADPS2) |
8               (1 << ADPS1) |
9               (1 << ADPS0);
10 }
```

Spustenie merania

Program nastaví `ADSC`. Po dokončení prevodu sa vykoná `ISR(ADC_vect)`, ak sú zároveň globálne prerušenia povolené pomocou `sei()`.

Spracovanie nameraných údajov

Priemerovanie, periodické vzorkovanie a kvalita merania

Prečo jednotlivé merania kolíšu

Aj pri konštantnom vstupnom napätí nemusí ADC vracať stále rovnaký kód.

Zdroje zmien

- šum snímača a napájania,
- digitálne rušenie mikrokontroléra,
- nestabilita referencie,
- kvantizačný krok ADC.

Typický prejav

Namiesto stálej hodnoty 512 môžeme pozorovať napríklad

$$511,\ 512,\ 513,\ 512,\ 511, \ldots$$

Jednoduchý aritmetický priemer z N meraní:

$$\bar{x} = \frac{1}{N} \sum_{k=1}^{N} x_k.$$

Čo priemerovanie robí

Potláča náhodné rýchle zmeny výsledku. Neodstraňuje systematickú chybu referencie, nesprávnu kalibráciu ani pomalý drift snímača.

Funkcia pre priemerovanie vzoriek

```
uint16_t adc_read_average(uint8_t channel,
                          uint8_t samples)
{
    uint32_t sum = 0;

    for (uint8_t i = 0; i < samples; i++)
    {
        sum += adc_read(channel);
    }

    return (uint16_t)(sum / samples);
}
```

Prečo je súčet 32-bitový

Pri sčítaní viacerých 10-bitových vzoriek môže súčet prekročiť rozsah 16-bitovej aritmetiky, najmä pri väčšom počte vzoriek.

Koľko vzoriek má zmysel priemerovať

Neexistuje univerzálne číslo. Voľba závisí od dynamiky meraného procesu.

Viac vzoriek

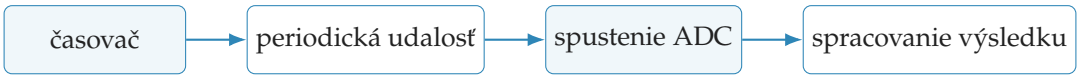
- hladší výsledok,
- menší vplyv náhodného šumu,
- pomalšia reakcia na reálnu zmenu signálu.

Menej vzoriek

- rýchlejšia odozva,
- väčšie kolísanie výsledku,
- nižšia výpočtová záťaž.

Periodické vzorkovanie pomocou časovača

Pre meranie s presne definovanou periódou nie je vhodné vytvárať časovanie iba pomocou dlhých blokujúcich oneskorení.



Nadviazanie na časovače

ADC možno automaticky spúšťať aj udalosťou časovača. Vtedy je vzorkovacia perióda určená hardvérom, nie časom vykonávania hlavnej slučky.

Výber vzorkovacej frekvencie podľa snímača

ADC môže byť rýchlejší než samotný snímač.

Príklad

Ak analógový snímač aktualizuje svoj výstup približne 60-krát za sekundu, meranie ADC rýchlosťou niekoľko tisíc vzoriek za sekundu nevytvorí tisíce nezávislých nových informácií o meranej veličine.

Zásada

Vzorkovaciu frekvenciu volíme podľa najrýchlejších zmien meraného signálu. Musí byť dostatočne vysoká na zachytenie jeho dynamiky, ale vyššia frekvencia než dokáže poskytnúť samotný snímač už nemusí priniesť novú informáciu.
V praxi sa vzorkovacia frekvencia často volí približne 5–10× vyššia než najvyššia významná frekvencia meraného signálu; $2f_{max}$ predstavuje iba teoretické minimum.

Presnosť nie je to isté ako rozlíšenie

Rozlíšenie

10 bitov znamená 1024 digitálnych kódov. Hovorí, aký jemný môže byť ideálny krok výsledku.

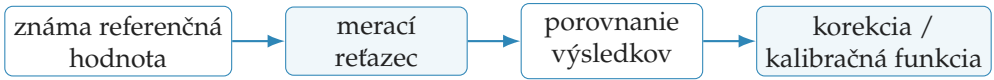
Presnosť

Vyjadruje, ako blízko je nameraný výsledok skutočnej hodnote. Ovplyvňujú ju referencia, offset, nelinearita, šum, zapojenie a snímač.

Dôsledok

Zobrazenie na viac desatinných miest ešte neznamená, že meranie má takúto fyzikálnu presnosť.

Kalibrácia meracieho reťazca



Prečo kalibrovať

Presnosť merania závisí od celého meracieho reťazca. Odchýlku môže spôsobovať snímač, referenčné napätie, analógové zapojenie aj samotný ADC.

Ako sa chyba znižuje

Meraná hodnota sa porovná so známou referenčnou hodnotou a z rozdielu sa určí korekcia, ktorá sa následne použije pri výpočte výsledku merania.

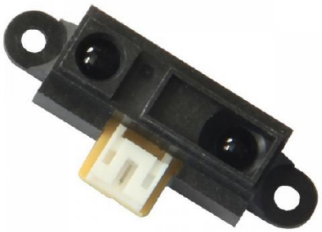
Aplikácia: analógový snímač vzdialenosti

Sharp GP2Y0A41SK0F ako príklad nelineárneho analógového snímača

Sharp GP2Y0A41SK0F

Základné vlastnosti

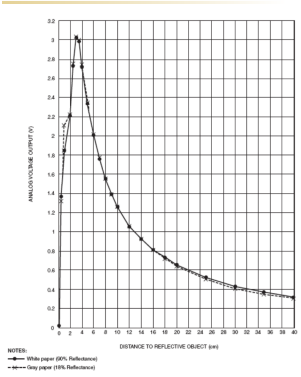
- optický snímač vzdialenosti,
- typický pracovný rozsah približne 4–30 cm,
- napájanie približne 4,5–5,5 V,
- analógové výstupné napätie,
- typická perióda merania približne 16,5 ms,
- typický priemerný odber približne 12 mA.



Význam pre ADC

Závislosť výstupného napätia od vzdialenosti je výrazne nelineárna. ADC vytvorí číselný kód, ale vzdialenosť musí dopočítať program z charakteristiky alebo kalibrácie.

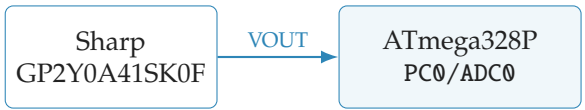
Prenosová charakteristika snímača



Pozorovanie

- výstupné napätie neklesá lineárne so vzdialenosťou,
- rovnaká zmena napätia neznamená v celom rozsahu rovnakú zmenu vzdialenosti,
- pri malej vzdialenosti je charakteristika strmšia než pri veľkej.

Zapojenie snímača k ADC0



Merací reťazec

Snímač vytvorí napätie V_{OUT}. ADC z neho vytvorí kód 0 . . . 1023. Program potom pomocou kalibračnej charakteristiky určí vzdialenosť.

Prečo nestačí lineárny prepočet

Nesprávna predstava:

$$d = a \cdot ADC + b$$

Nelineárna charakteristika

Jedna lineárna rovnica môže byť prijateľná iba na malom úseku charakteristiky. Pre celý rozsah by chyba výrazne rástla.

Možnosti spracovania

- aproximačná nelineárna rovnica - správanie snímača sa opíše rovnicou napr. polynómom alebo inou funkciou.
- kalibračná tabuľka a interpolácia - uloží sa viac známych dvojíc meraná hodnota skutočná hodnota a medzi nimi sa výsledok dopočíta.
- po častiach lineárna aproximácia - celý nelineárny priebeh sa rozdelí na menšie úseky a každý sa nahradí jednoduchou priamkou.

Kalibračná tabuľka

Vzdialenosť [cm]	Výstupné napätie [V]	ADC kód pri 5 V
5	približne 2,7	približne 553
10	približne 1,4	približne 287
15	približne 0,9	približne 184
20	približne 0,65	približne 133
30	približne 0,42	približne 86

Poznámka

Uvedené hodnoty sú orientačné podľa charakteristiky snímača. Pre presnejšie meranie sa kalibračné body získajú experimentálne pre konkrétny kus a konkrétny merací reťazec.

Ak nameraná hodnota leží medzi dvoma kalibračnými bodmi (x_1, d_1) a (x_2, d_2), možno na krátkom úseku použiť lineárnu interpoláciu:

$$d \approx d_1 + (x - x_1)\frac{d_2 - d_1}{x_2 - x_1}$$

Význam

Celá nelineárna krivka sa rozdelí na viac krátkych úsekov. Každý úsek sa lokálne aproximuje priamkou.

Meranie snímača a výpis cez USART

```c
int main(void)
{
    char text[48];
    uint16_t adc_value;
    uint32_t voltage_mv;
    adc_init();
    usart0_init();
    while (1)
    {
        adc_value = adc_read_average(0, 8);
        voltage_mv = ((uint32_t)adc_value * 5000UL) / 1024UL;
        snprintf(text, sizeof(text),
                 "ADC=%u U=%lu mV\r\n",
                 adc_value, voltage_mv);
        usart0_send_string(text);
        _delay_ms(100);
    }
}
```

Zhrnutie

Od analógového vstupu po použiteľnú meranú veličinu

Postup návrhu analógového merania

1. určiť rozsah a dynamiku analógového signálu,
2. zvoliť vhodné referenčné napätie,
3. overiť zdrojovú impedanciu a analógové zapojenie,
4. nastaviť hodinový signál ADC,
5. určiť vzorkovaciu frekvenciu,
6. zvoliť jednorazový prevod, automatické spúšťanie alebo prerušenie,
7. správne spracovať výsledok v jazyku C,
8. podľa potreby filtrovať a kalibrovať celý merací reťazec.

Ďakujem za pozornosť